

# Примеры использования ассемблера GNU

*GAS, ассемблер GNU, является стандартным ассемблером операционной системы GNU. Он работает на многих различных архитектурах и поддерживает несколько синтаксисов языка ассемблера. Однако эти примеры относятся только к операционным системам, использующим ядро Linux и процессор x86-64.*

## СОДЕРЖАНИЕ

Начало работы • Работа с библиотекой C • Соглашения о вызове функций для 64-битного кода на C • Сочетание C и языка ассемблера • Аргументы командной строки • Инструкции с плавающей запятой • Разделы данных • Рекурсия • SIMD-параллелизм • Арифметика с насыщением • Локальные переменные и кадры стека

## Начиная

Вот традиционная программа "Hello World", использующая системные вызовы Linux, для 64-битной установки:

привет.s

```
# -----  
# Writes "Hello, World" to the console using only system calls. Runs on 64-bit Linux only.  
# To assemble and run:  
#  
# gcc -c hello.s && ld hello.o && ./a.out  
#  
# or  
#  
# gcc -nostdlib hello.s && ./a.out  
# -----  
  
    .global _start  
  
    .text  
_start:  
    # write(1, message, 13)  
    mov     $1, %rax           # system call 1 is write  
    mov     $1, %rdi           # file handle 1 is stdout  
    mov     $message, %rsi     # address of string to output
```

```

mov    $13, %rdx          # number of bytes
syscall                          # invoke operating system to do the write

# exit(0)
mov    $60, %rax          # system call 60 is exit
xor    %rdi, %rdi         # we want return code 0
syscall                          # invoke operating system to exit
message:
    .ascii "Hello, world\n"

```

```

$gcc -c hello.s && ld hello.o && ./a.out
Привет, мир

```

Если вы используете другую операционную систему, например macOS или Windows, то номера системных вызовов и используемые регистры, скорее всего, будут отличаться.

## Работа с библиотекой C

Как правило, вам понадобится использовать библиотеку на языке C. Вот ещё раз пример "Hello World":

**хола.с**

```

# -----
# Writes "Hola, mundo" to the console using a C library. Runs on Linux or any other system
# that does not use underscores for symbols in its C library. To assemble and run:
#
#     gcc hola.s && ./a.out
# -----

    .global main

    .text

main:                                # This is called by C library's startup code
    mov    $message, %rdi          # First integer (or pointer) parameter in %rdi
    call   puts                   # puts(message)
    ret                             # Return to C library code

message:
    .asciz "Hola, mundo"          # asciz puts a 0 byte at the end

```

```

$gcc hola.s && ./a.out
Привет, мир

```

# Соглашения о вызове функций для 64-битного кода на языке C.

Соглашения о вызове функций в 64-битной среде немного подробнее, и они полностью объяснены в [справочнике AMD64 ABI](#) . Вы также можете найти информацию о них в [Википедии](#) . Наиболее важные моменты (опять же, для 64-битного Linux, а не для Windows):

- Слева направо передавайте столько параметров, сколько поместится в регистрах. Порядок выделения регистров следующий:
  - Для целых чисел и указателей: `rdi, rsi, rdx, rcx, r8, r9`.
  - Для чисел с плавающей запятой (`float, double`): `xmm0, xmm1, xmm2, xmm3, xmm4, xmm5, xmm6, xmm7`
- Дополнительные параметры помещаются в стек справа налево и удаляются вызывающей стороной после вызова функции.
- После передачи параметров выполняется инструкция вызова, поэтому, когда вызываемая функция получает управление, адрес возврата находится по адресу `(%rsp)` , первый параметр в памяти – по адресу `8(%rsp)` , и так далее.
- **Указатель стека `%RSP` должен быть выровнен по 16-байтовой границе перед вызовом функции** . Хорошо, но в процессе вызова адрес возврата (8 байт) помещается в стек, поэтому, когда функция получает управление, `%rsp` оказывается не выровненным. Вам придется самостоятельно создать это дополнительное пространство, поместив что-нибудь в стек или вычтя 8 из `%rsp`.
- Единственные регистры, которые вызываемая функция обязана сохранять (регистры, сохраняемые при вызове), это: `rbp, rbx, r12, r13, r14, r15`. Все остальные регистры могут быть изменены вызываемой функцией.
- Предполагается также, что вызываемая функция сохранит управляющие биты регистра `CR2` и управляющее слово `xCR2`, но инструкции `xCR2` редко встречаются в 64-битном коде, поэтому вам, вероятно, не стоит об этом беспокоиться.
- Целые числа возвращаются в формате `rax` или `rdi:rdi`, а значения с плавающей запятой – в формате `xmm0` или `xmm1:xmm0`.

Эта программа выводит первые несколько чисел Фибоначчи, иллюстрируя, как необходимо сохранять и восстанавливать данные в регистрах:

```
фиб.с
# -----
# A 64-bit Linux application that writes the first 90 Fibonacci numbers. It
# needs to be linked with a C library.
#
# Assemble and Link:
#   gcc fib.s
# -----
```

```

.global main

.text

main:
    push    %rbx                # we have to save this since we use it

    mov     $90, %ecx           # ecx will countdown to 0
    xor     %rax, %rax          # rax will hold the current number
    xor     %rbx, %rbx          # rbx will hold the next number
    inc     %rbx                # rbx is originally 1

print:
    # We need to call printf, but we are using eax, ebx, and ecx.  printf
    # may destroy eax and ecx so we will save these before the call and
    # restore them afterwards.

    push    %rax                # caller-save register
    push    %rcx                # caller-save register

    mov     $format, %rdi       # set 1st parameter (format)
    mov     %rax, %rsi          # set 2nd parameter (current_number)
    xor     %rax, %rax          # because printf is varargs

    # Stack is already aligned because we pushed three 8 byte registers
    call    printf              # printf(format, current_number)

    pop     %rcx                # restore caller-save register
    pop     %rax                # restore caller-save register

    mov     %rax, %rdx          # save the current number
    mov     %rbx, %rax          # next number is now current
    add     %rdx, %rbx          # get the new next number
    dec     %ecx                # count down
    jnz     print               # if not done counting, do some more

    pop     %rbx                # restore rbx before returning
    ret

format:
    .asciz  "%20ld\n"

```

```
$gcc fib.s && ./a.out
```

```
0
```

```
1
```

```
1
```

```
2
```

```
3
```

```
...
```

```
420196140727489673
```

```
679891637638612258
```

```
1100087778366101931
1779979416004714189
```

# Сочетание языка C и ассемблера

Эта 64-битная программа представляет собой очень простую функцию, которая принимает три 64-битных целочисленных параметра: они помещаются в стек, так что при входе в функцию они будут находиться в `rdi`, `rsi` и `rdx` соответственно. Возвращаемое значение является целым числом, поэтому оно возвращается в `rax`.

## maxofthree.s

```
# -----
# A 64-bit function that returns the maximum value of its three 64-bit integer
# arguments. The function has signature:
#
# int64_t maxofthree(int64_t x, int64_t y, int64_t z)
#
# Note that the parameters have already been passed in rdi, rsi, and rdx. We
# just have to return the value in rax.
# -----

        .globl  maxofthree

        .text
maxofthree:
        mov     %rdi, %rax           # result (rax) initially holds x
        cmp     %rsi, %rax           # is x less than y?
        cmovl   %rsi, %rax           # if so, set result to y
        cmp     %rdx, %rax           # is max(x,y) less than z?
        cmovl   %rdx, %rax           # if so, set result to z
        ret                                # the max will be in eax
```

Вот программа на языке C, которая вызывает функцию языка ассемблера.

## callmaxofthree.c

```
/*
 * callmaxofthree.c
 *
 * A small program that illustrates how to call the maxofthree function we wrote in
 * assembly language.
 */

#include <stdio.h>
#include <inttypes.h>
```

```
int64_t maxofthree(int64_t, int64_t, int64_t);

int main() {
    printf("%ld\n", maxofthree(1, -4, -7));
    printf("%ld\n", maxofthree(2, -6, 1));
    printf("%ld\n", maxofthree(2, 3, 1));
    printf("%ld\n", maxofthree(-2, 4, 3));
    printf("%ld\n", maxofthree(2, -6, 5));
    printf("%ld\n", maxofthree(2, 4, 6));
    return 0;
}
```

Для сборки, компоновки и запуска этой двухчастной программы:

```
$gcc -std=c99 callmaxofthree.c maxofthree.s && ./a.out
1
2
3
4
5
6
```

## Аргументы командной строки

Вы знаете, что в языке C `main` это просто обычная функция, и у неё есть пара собственных параметров:

```
int main(int argc, char** argv)
```

Вот программа, которая использует этот факт для простого вывода аргументов командной строки в другую программу, по одному на строку:

ЭХО.C

```
# -----
# A 64-bit program that displays its commandline arguments, one per line.
#
# On entry, %rdi will contain argc and %rsi will contain argv.
# -----

        .global main

        .text
main:
        push    %rdi                # save registers that puts uses
        push    %rsi
        sub     $8, %rsp            # must align stack before call
```

```

mov    (%rsi), %rdi    # the argument string to display
call   puts            # print it

add     $8, %rsp        # restore %rsp to pre-aligned value
pop     %rsi            # restore registers puts used
pop     %rdi

add     $8, %rsi        # point to next argument
dec     %rdi            # count down
jnz     main            # if not done counting keep going

ret

```

format:

```
.asciz    "%s\n"
```



```

$gcc echo.s && ./a.out 25782 dog huh $$
./a.out
25782
собака
хм
9971
$gcc echo.s && ./a.out 25782 dog huh '$$'
./a.out
25782
собака
хм
$$

```

Обратите внимание, что с точки зрения библиотеки C аргументы командной строки всегда являются строками. Если вы хотите обрабатывать их как целые числа, вызовите метод `atoi`. Вот небольшая программа для вычисления  $x^y$ . Еще одна особенность этого примера заключается в том, что он показывает, как ограничить значения 32-битными.

#### МОЩНОСТЬ.C

```

# -----
# A 64-bit command line application to compute x^y.
#
# Syntax: power x y
# x and y are integers
# -----

.global main

.text

main:
    push    %r12        # save callee-save registers
    push    %r13
    push    %r14
    # By pushing 3 registers our stack is already aligned for calls

```

```

    cmp    $3, %rdi          # must have exactly two arguments
    jne    error1

    mov     %rsi, %r12        # argv

# We will use ecx to count down from the exponent to zero, esi to hold the
# value of the base, and eax to hold the running product.

    mov     16(%r12), %rdi    # argv[2]
    call    atoi              # y in eax
    cmp     $0, %eax          # disallow negative exponents
    jl      error2
    mov     %eax, %r13d       # y in r13d

    mov     8(%r12), %rdi     # argv
    call    atoi              # x in eax
    mov     %eax, %r14d       # x in r14d

    mov     $1, %eax          # start with answer = 1
check:
    test    %r13d, %r13d      # we're counting y down to 0
    jz      gotit             # done
    imul    %r14d, %eax        # multiply in another x
    dec     %r13d
    jmp     check
gotit:
                                # print report on success
    mov     $answer, %rdi
    movslq  %eax, %rsi
    xor     %rax, %rax
    call    printf
    jmp     done
error1:
                                # print error message
    mov     $badArgumentCount, %edi
    call    puts
    jmp     done
error2:
                                # print error message
    mov     $negativeExponent, %edi
    call    puts
done:
                                # restore saved registers
    pop     %r14
    pop     %r13
    pop     %r12
    ret

answer:
    .asciz  "%d\n"
badArgumentCount:
    .asciz  "Requires exactly two arguments\n"

```

negativeExponent:

```
.asciz "The exponent may not be negative\n"
```

```
$. /power 2 19
524288
$. /power 3 -8
Показатель степени не может быть отрицательным.
$. /power 1 500
1
```

*Упражнение : Перепишите этот пример, используя 64-битные числа повсюду. Вам также потребуется перейти от `atoi` к `strtol`.*

## Инструкции для работы с числами с плавающей запятой

Аргументы с плавающей запятой записываются в регистры `xmm`. Вот простая функция для суммирования значений в массиве типа `double`:

сумма.с

```
# -----
# A 64-bit function that returns the sum of the elements in a floating-point
# array. The function has prototype:
#
# double sum(double[] array, unsigned length)
# -----

.global    sum
.text

sum:
    xorpd    %xmm0, %xmm0        # initialize the sum to 0
    cmp     $0, %rsi             # special case for length = 0
    je      done

next:
    addsd    (%rdi), %xmm0        # add in the current array element
    add     $8, %rdi              # move to next array element
    dec     %rsi                 # count down
    jnz     next                 # if not done counting, continue

done:
    ret                                # return value already in xmm0
```

Программа `АС`, которая так это называет:

callsum.c

```

/*
 * callsum.c
 *
 * Illustrates how to call the sum function we wrote in assembly language.
 */

#include <stdio.h>

double sum(double[], unsigned);

int main() {
    double test[] = {
        40.5, 26.7, 21.9, 1.5, -40.5, -23.4
    };
    printf("%20.7f\n", sum(test, 6));
    printf("%20.7f\n", sum(test, 2));
    printf("%20.7f\n", sum(test, 0));
    printf("%20.7f\n", sum(test, 3));
    return 0;
}

```

```

$gcc callsum.c sum.s && ./a.out
26.7000000
67.2000000
0.0000000
89.1000000

```

## Разделы данных

В большинстве операционных систем текстовый раздел доступен только для чтения, поэтому вам может понадобиться раздел данных. В большинстве операционных систем раздел данных предназначен только для инициализированных данных, а для неинициализированных данных существует специальный раздел `.bss`. Вот программа, которая усредняет аргументы командной строки, которые должны быть целыми числами, и отображает результат в виде числа с плавающей запятой.

`среднее.c`

```

# -----
# 64-bit program that treats all its command line arguments as integers and
# displays their average as a floating point number. This program uses a data
# section to store intermediate results, not that it has to, but only to
# illustrate how data sections are used.
# -----

.globl main

```

```

        .text

main:
    dec    %rdi                # argc-1, since we don't count program name
    jz     nothingToAverage
    mov    %rdi, count        # save number of real arguments

accumulate:
    push   %rdi                # save register across call to atoi
    push   %rsi
    mov    (%rsi,%rdi,8), %rdi  # argv[rdi]
    call   atoi                # now rax has the int value of arg
    pop    %rsi                # restore registers after atoi call
    pop    %rdi
    add    %rax, sum            # accumulate sum as we go
    dec    %rdi                # count down
    jnz    accumulate         # more arguments?

average:
    cvtsi2sd sum, %xmm0
    cvtsi2sd count, %xmm1
    divsd  %xmm1, %xmm0        # xmm0 is sum/count
    mov    $format, %rdi       # 1st arg to printf
    mov    $1, %rax            # printf is varargs, there is 1 non-int argument

    sub    $8, %rsp            # align stack pointer
    call   printf              # printf(format, sum/count)
    add    $8, %rsp            # restore stack pointer

    ret

nothingToAverage:
    mov    $error, %rdi
    xor    %rax, %rax
    call   printf
    ret

        .data
count: .quad 0
sum: .quad 0
format: .asciz "%g\n"
error: .asciz "There are no command line arguments to average\n"

```

## Рекурсия

Возможно, это покажется удивительным, но для реализации рекурсивных функций не требуется ничего необычного. Нужно лишь, как обычно, следить за сохранением регистров. Вот пример. На языке C:

```
uint64_t factorial ( unsigned n ) { return ( n <= 1 ) ? 1 : n * factorial ( n - 1 ); }
```

### факториал.с

```
# -----
# A 64-bit recursive implementation of the function
#
#   uint64_t factorial(unsigned n)
#
# implemented recursively
# -----

        .globl factorial

        .text
factorial:
        cmp     $1, %rdi           # n <= 1?
        jnbe    L1                # if not, go do a recursive call
        mov     $1, %rax           # otherwise return 1
        ret

L1:
        push    %rdi              # save n on stack (also aligns %rsp!)
        dec     %rdi              # n-1
        call    factorial         # factorial(n-1), result goes in %rax
        pop     %rdi              # restore n
        imul    %rdi, %rax        # n * factorial(n-1), stored in %rax
        ret
```

Пример вызывающего абонента:

### callfactorial.c

```
/*
 * An application that illustrates calling the factorial function defined elsewhere.
 */

#include <stdio.h>
#include <inttypes.h>

uint64_t factorial(unsigned n);

int main() {
    for (unsigned i = 0; i < 20; i++) {
        printf("factorial(%2u) = %lu\n", i, factorial(i));
    }
}
```

# SIMD-параллелизм

Регистры XMM могут выполнять арифметические операции над значениями с плавающей запятой по одной операции за раз или по нескольким операциям одновременно. Операции имеют следующий вид:

операция `xmmregister_or_memorylocation, xmmregister`

Для сложения чисел с плавающей запятой используются следующие инструкции:

`addpd` – выполнить 2 сложения с двойной точностью

`addps` – выполняет всего одно сложение с двойной точностью, используя младшие 64 бита регистра.

`addsd` – выполнить 4 сложения одинарной точности

`addss` – выполнить только одно сложение одинарной точности, используя младшие 32 бита регистра.

TODO - показать функцию, которая обрабатывает массив чисел с плавающей запятой по 4 за раз.

# Насыщенная арифметика

Регистры XMM также могут выполнять арифметические операции над целыми числами. Инструкции имеют следующий вид:

операция `xmmregister_or_memorylocation, xmmregister`

Для сложения целых чисел используются следующие инструкции:

`paddb` – выполняет сложение 16 байтов

`paddw` – сложить 8 слов

`paddd` – выполнить 4 сложения `dword`

`paddq` – выполнить 2 добавления `qword`

`paddsb` – сложение 16 байтов с насыщением знака (80..7F)

`paddsw` – сложение 8 слов с беззнаковой насыщенностью (8000..7FFF)

`paddusb` – выполняет сложение 16 байтов с беззнаковым насыщением (00..FF)

`paddusw` – выполнить сложение 8 слов с беззнаковой насыщенностью (00..FFFF)

TODO - ПОКАЖИТЕ ПРИМЕР

# Локальные переменные и стековые кадры

Для начала, пожалуйста, прочтите [статью Эли Бендерски](#). Этот обзор более полный, чем мои краткие заметки.

При вызове функции вызывающая сторона сначала помещает параметры в соответствующие регистры, а затем выдает инструкцию `call`. Дополнительные параметры, помимо тех, которые охватываются регистрами, помещаются в стек перед вызовом. Инструкция вызова помещает адрес возврата на вершину стека. Таким образом, если у вас есть функция

```
long example ( long x , long y ) { long a , b , c ;      b = 7 ; return x * b + y ; }
```

Затем при входе в функцию переменная `x` будет находиться в `%edi`, `y` – в `%esi`, а адрес возврата – на вершине стека. Куда же можно поместить локальные переменные? Простой вариант – в сам стек, хотя, если у вас достаточно регистров, используйте их.

Если вы работаете на машине, которая поддерживает стандартный ABI, вы можете оставить `%rsp` как есть и получать доступ к "дополнительным параметрам" и локальным переменным непосредственно из `%rsp`, например:

```

      +-----+
rsp-24 | a |
      +-----+
rsp-16 | b |
      +-----+
rsp-8  | c |
      +-----+
rsp   | retaddr |
      +-----+
rsp+8 | звонящего |
      | стек |
      | рамка |
      | ... |
      +-----+
```

Таким образом, наша функция выглядит следующим образом:

```
.text .globl example example :      movl    $7 , -16 ( % rsp )      mov     %rdi , %rax
```

Если наша функция совершит ещё один вызов, вам придётся скорректировать `%rsp`, чтобы освободить место в этот момент.

В Windows эта схема недоступна, потому что при возникновении прерывания все данные выше указателя стека будут заняты. В большинстве других операционных систем этого не происходит, поскольку существует «красная зона» в 128 байт после указателя стека, которая защищена от подобных событий. В этом случае вы можете немедленно освободить место в стеке:

пример :

```
sub    $24 , %rsp
```

Таким образом, наш стек выглядит следующим образом:

```

      +-----+
rsp | a |
      +-----+
rsp+8 | b |
      +-----+
rsp+16 | c |
      +-----+
rsp+24 | retaddr |
      +-----+
rsp+32 | звонящего |
      | стек |
      | рамка |
      | ... |
      +-----+

```

Вот как выглядит функция сейчас. Обратите внимание, что перед возвратом необходимо заменить указатель стека!

```
.text .globl example example :      sub    $24 , %rsp      movl    $7 , 8 (% rsp )      mo'
```